ROS NAVIGATION REPORT

Presenter : Hsiang-Ho Shih, Chia-Jui Yang
Professor : Prof. Dr. Alexander Ferrein
Date : 25 Jun 2025
Course : Roboterprogrammierung mit ROS

Table of contents

- Motivation
- Planner
  - Overall
  - 2D Planner
  - 3D Planner
- Conclusion
- Future Work
- Problem
- Reference

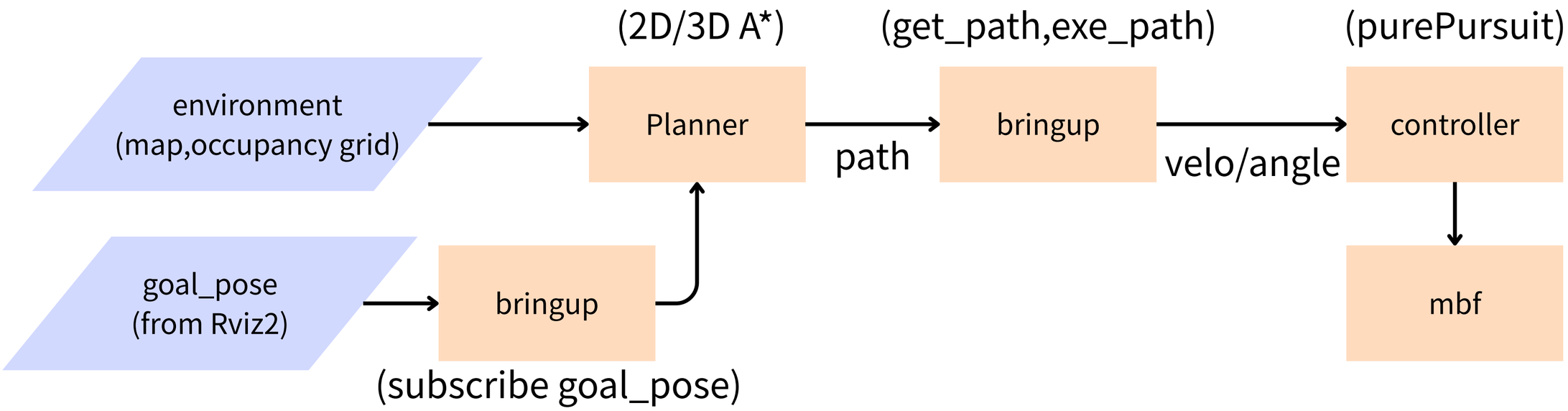
Motivation

- Robots must navigate over spaces with obstacle safely and efficiently.

- Off-the-shelf planners struggle with:
  - Octo navigation is more complex than 2D navigation
  - Situations requiring the robot to change its body posture

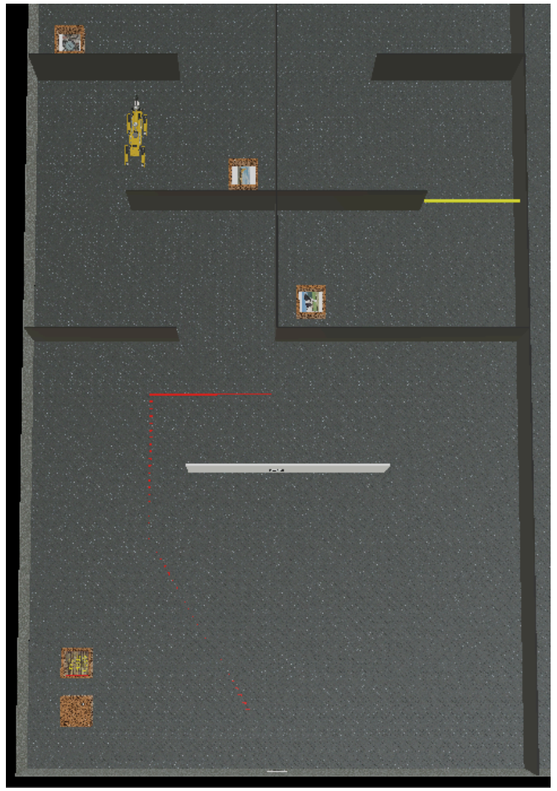
- Build 2-D / 3-D A* navigation that:
  - Produces smooth paths for robot
  - Plans collision-free trajectories through full 3-D voxel space when needed
  - Executes trajectories using a controller while dynamically adjusting body height

NAV flowchart



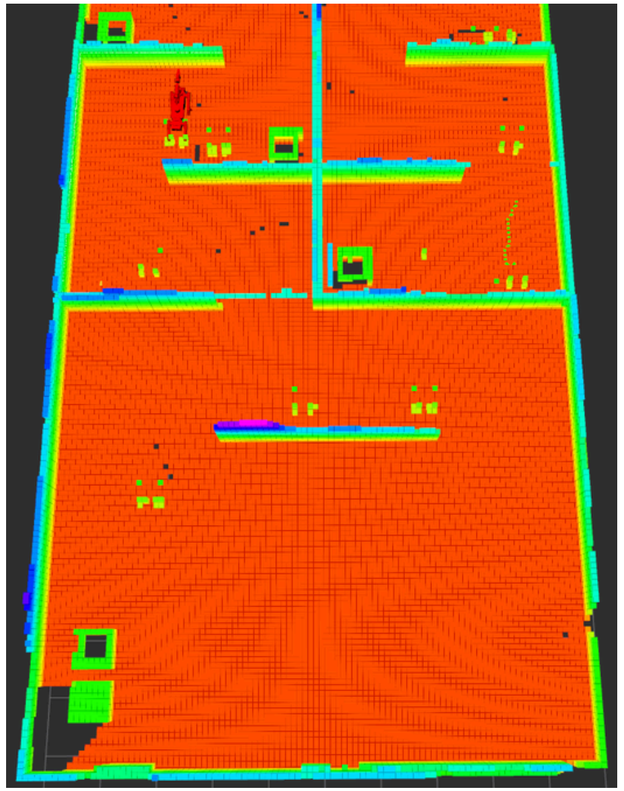
Map for Planner

environment



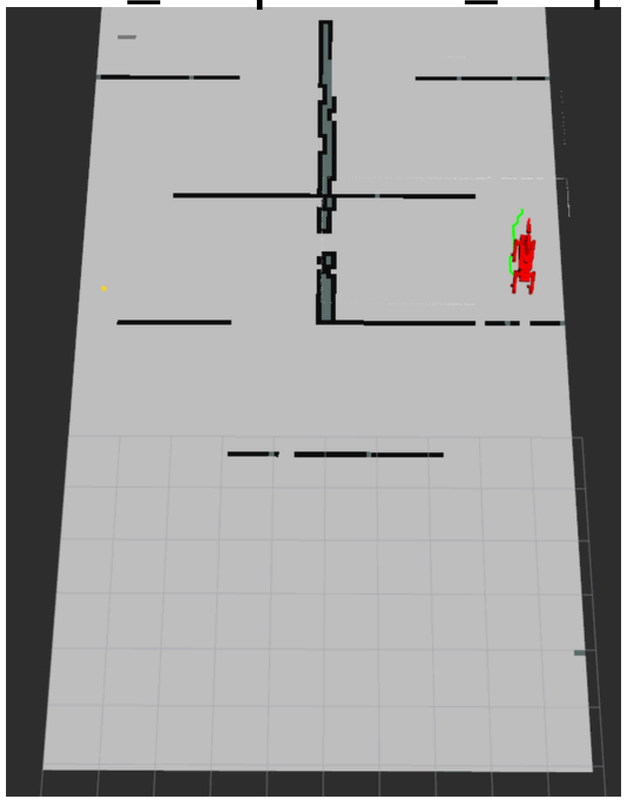
spot simulation

octomap (For 3D)



Voxel size:0.1m
Color transformer:axis
Using depth camera

map conversion (For 2D)
(octo_map->UGV_map)



Resolution:0.1m
max_slope_ugv: 0.7

2D nav planner ability

plan_3d_2d_path.py (Reference)

python code

A* path search

pointcloud_callback

astar_2d_planner.cpp

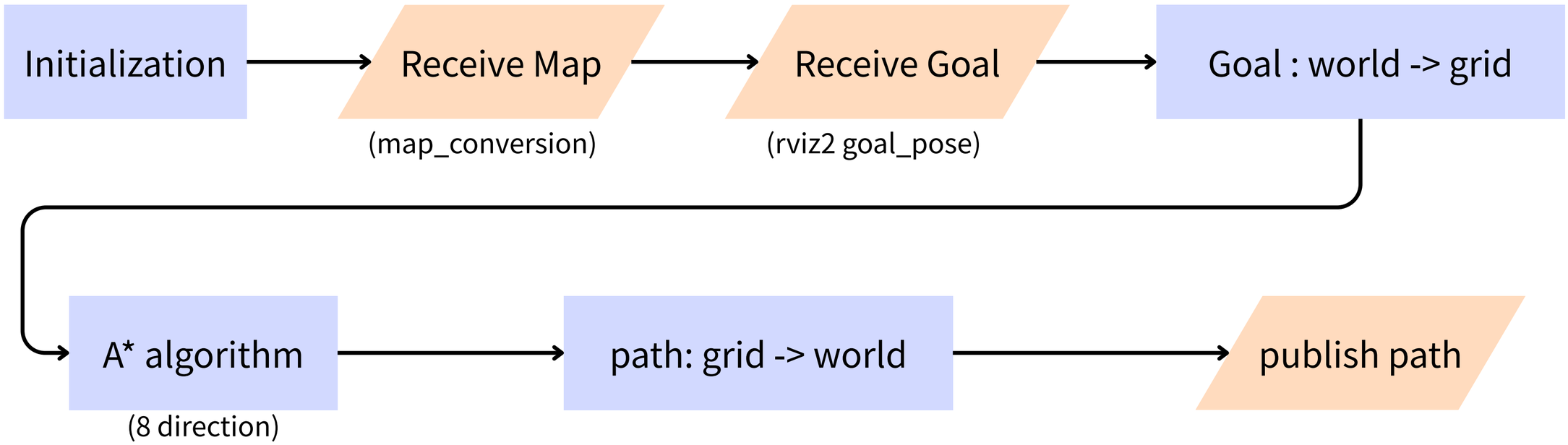
c++ code(.cpp, CmakeLists.txt,Package.xml)

A* path search

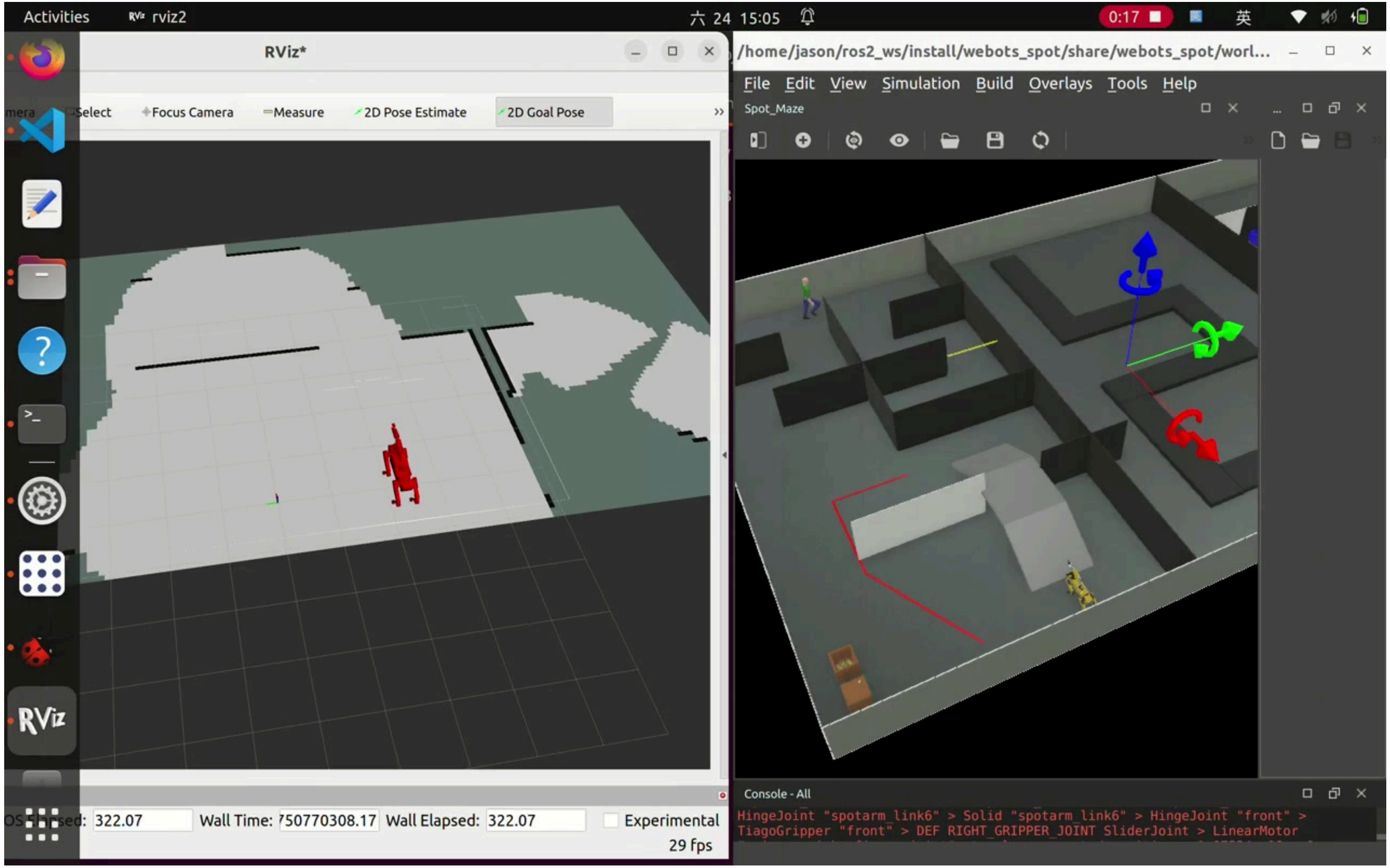
map_conversion package
[consider slope and height(z)]

exe_path and get_path(service client)
smoother path

2D nav planner flowchart

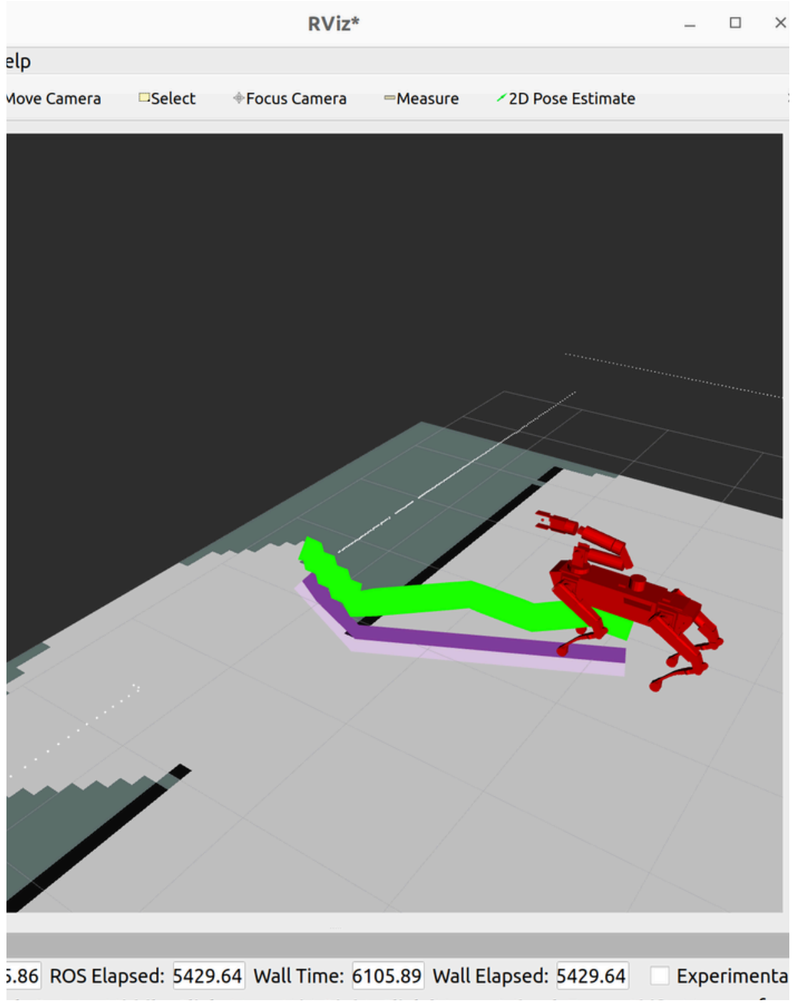


Experimental Results : 2D nav planner going on ramp



2D nav planner smooth path

- Vertex Pruning
  - Remove middle point if 3 points are aligned.
- Line-of-Sight Shortcut
  - Raycast from current to later node; remove intermediate points if no obstacles.
- Curve Interpolation
  - Generate smooth curve from points, resample evenly.



Green : Original path

Purple : Smooth Path

3D nav planner ability

2D planner

A* path search

map_conversion(from Occupancy grid)

detect strategy in 2D A*
(only Occupied)

3D planner

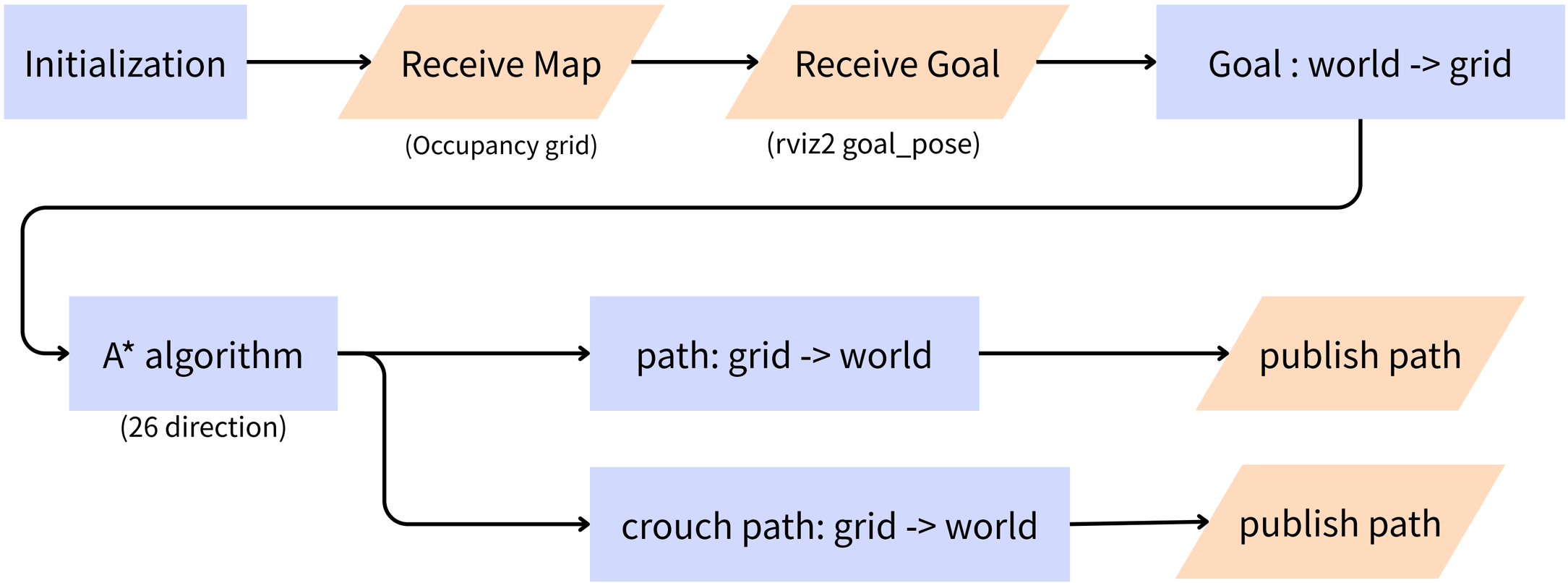
A* path search

Occupancy grid(from pointcloud2)

detect strategy in 3D A*
(CylinderCollision,Occupied,Occupied_above)

Crouch Algorithm

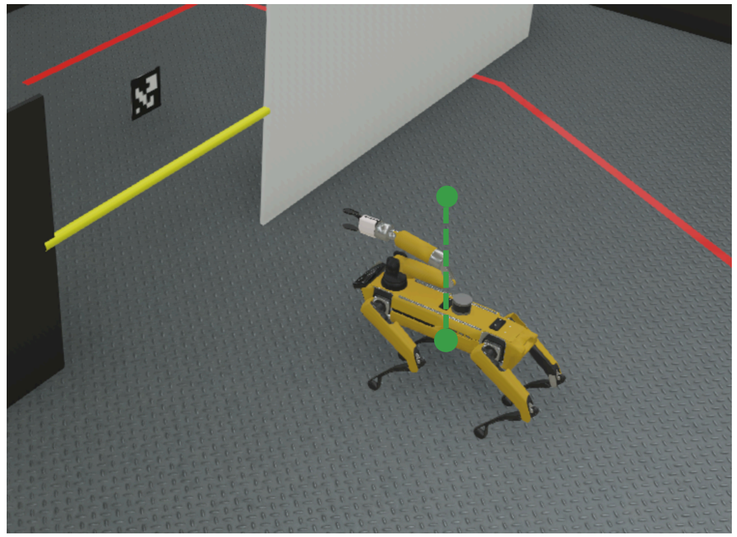
3D nav planner flowchart



detect strategy in 3D A*

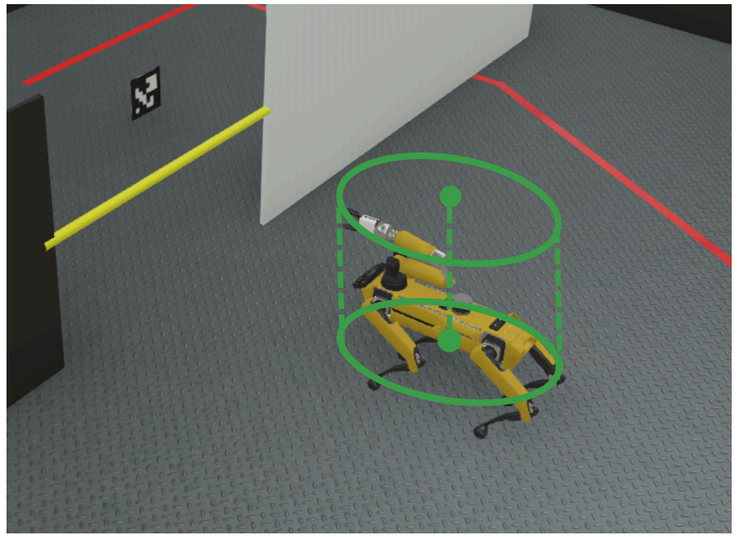
parameter in astar_octo_planner.h

hasNoOccupiedCellsAbove



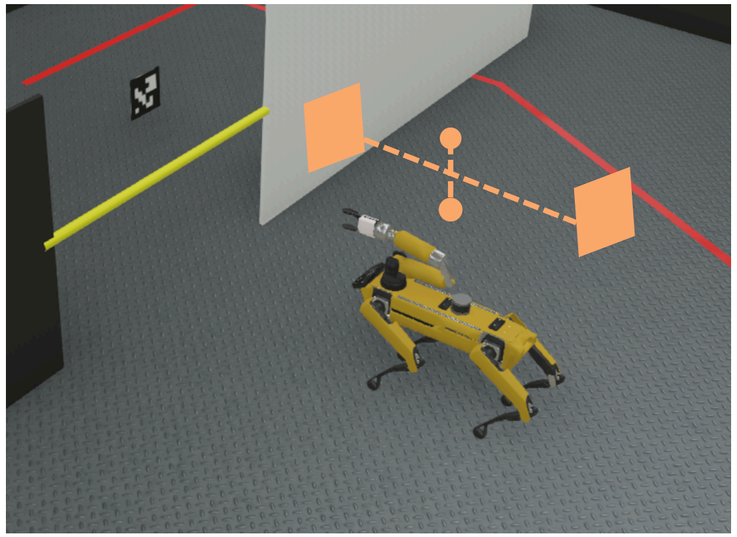
min_vertical_clearance_
max_vertical_clearance_

isCylinderCollisionFree



robot_radius_
min_vertical_clearance_
max_vertical_clearance_

needToSquat



squat_x
squat_y
max_vertical_clearance
max_vertical_squat_

detect strategy in 3D A*

feasible path

hasNoOccupiedCellsAbove

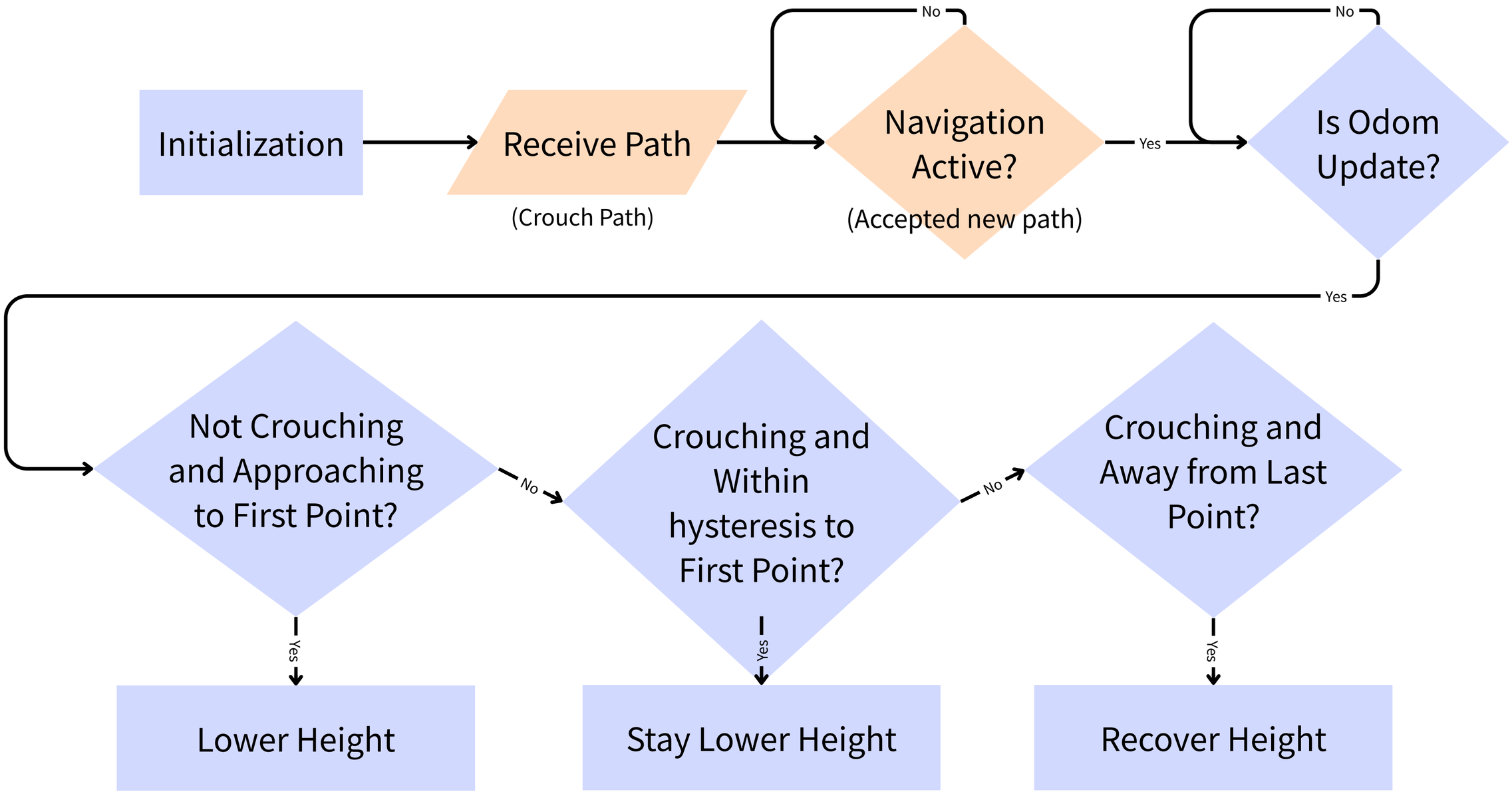
isCylinderCollisionFree

if robot crouched down, it can pass it

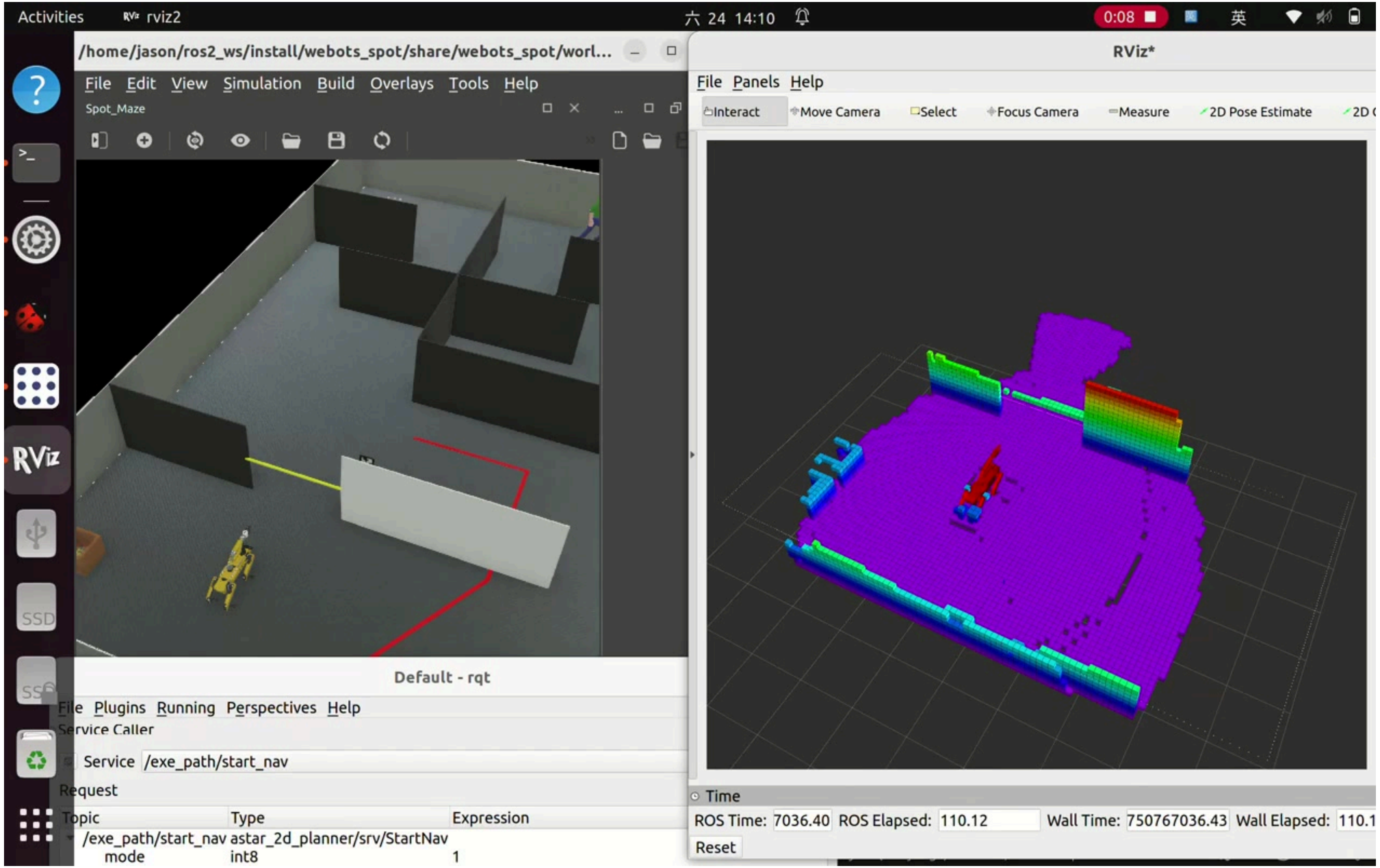
needToSquat

where should the robot lower height and recover path (start, end)

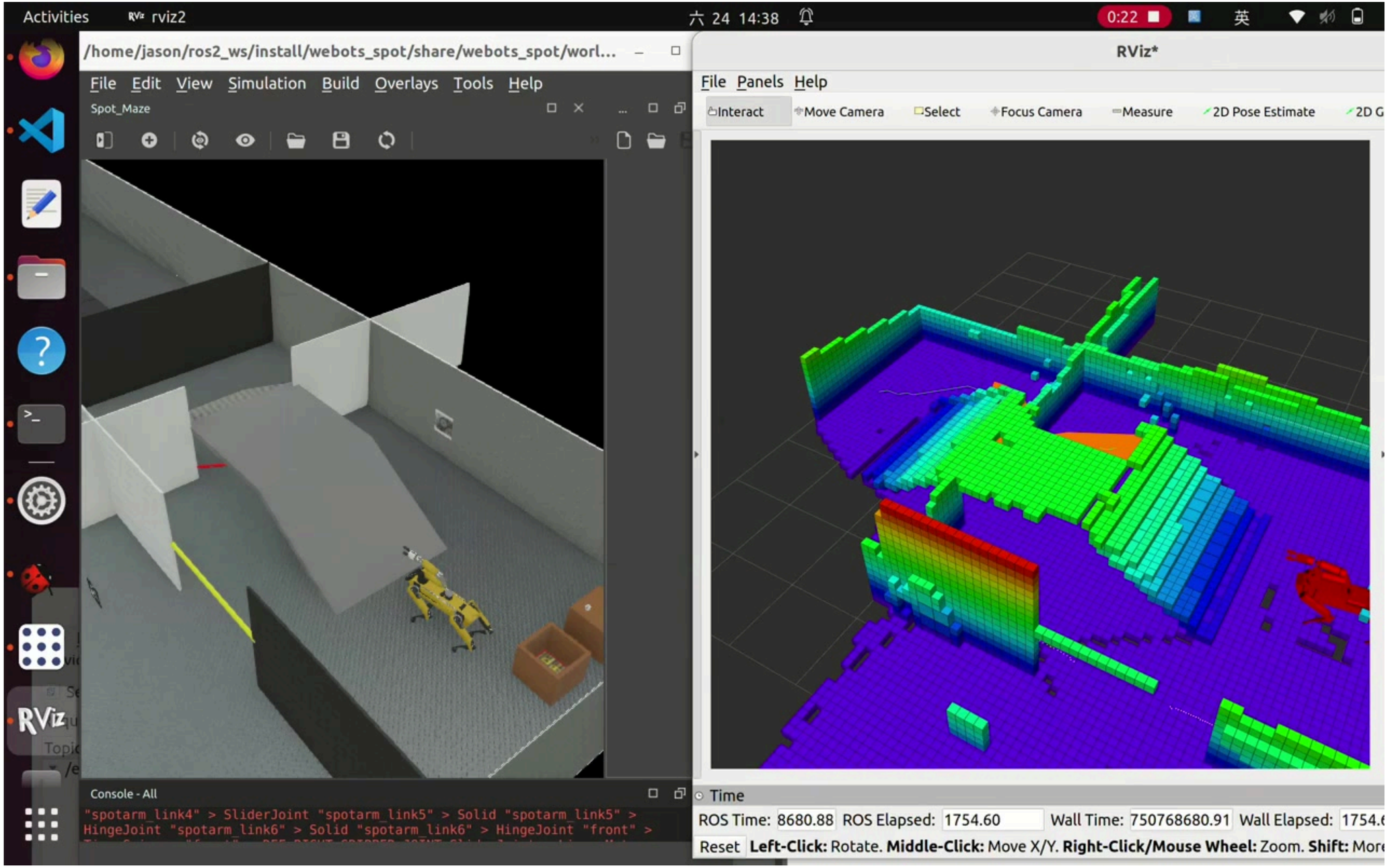
Crouch flowchart



Experimental Results : 3D nav planner go through bar



Experimental Results : 3D nav planner going on ramp



Conclusion

2D planner

- change 2D planner.py to C++ file(.cpp, Package.xml, CmakeList.txt)
- add move path,exe path, get path when we set the goal_point
- create the smoothed path, improve the time and more continuity

3D planner

- add detect strategy when we need to crouch
- add crouch algorithm when we get the crouch path

build package(bring up)

- include node for exe_path, get_path
- include node for sub_goal_pose, sub_crouch_path, service_set_height

Future Work

- Make code more efficient for better performance on robot.
- Try other algorithm beside a*.
- Add other control (such as mesh controller).
- Implain another planner (like rrt_exploration).

Problems

- Lack of expension of obstacle in 2D navigation.
- Noise from octo map will cause wrong result of obstacle detection.
- In Spot simulation robot walk unstable, even worse when crouching and climbing the ramp.

Reference

- https://github.com/RRL-ALeRT/octo_navigation
- https://github.com/RRL-ALeRT/octomap_mapping/tree/ros2
- https://github.com/LTU-RAI/Map-Conversion-3D-Voxel-Map-to-2D-Occupancy-Map